



UNIVERSIDAD DEL VALLE
ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Calculando el Plan de Riego Óptimo de una Finca

Autores:

Ervin Caravali Ibarra
ervin.caravali@correounivalle.edu.co

Camilo Urrea
camilo.urrea@correounivalle.edu.co

Asignatura:

Análisis de Algoritmos II

Cali, Colombia
2026

Índice

I.	Introducción y Contexto	2
I-A.	El Problema Real y la Importancia del Riego Óptimo	2
I-B.	Restricciones Prácticas y Estrés Hídrico	2
I-C.	Motivación Tecnológica e Impacto	2
II.	Formalización Matemática	2
II-A.	Definición del Espacio del Problema: La Finca	2
II-B.	La Programación de Riego (Permutación)	2
II-C.	Cálculo de Tiempos y Función de Costo Individual (CRF)	3
II-D.	Función de Costo Objetivo	3
III.	Naturaleza Combinatoria del Problema	3
IV.	Análisis Completo de Ejemplos y Funcionamiento Interno	3
IV-A.	Fuerza Bruta: El Árbol de Búsqueda (DFS)	3
IV-B.	Voraz: Matriz de Descarte Secuencial y Método Piloto	3
IV-C.	Programación Dinámica: Matriz de Ensamblaje Bottom-Up	4
V.	Validación Empírica	4
VI.	Algoritmo 1 — Fuerza Bruta	4
VI-A.	Idea Conceptual	4
VI-B.	Complejidad Computacional (Fuerza Bruta)	5
VI-C.	Consistencia y Desempate Lexicográfico	6
VI-D.	Demostración Matemática (Fuerza Bruta)	6
VII.	Algoritmo 2 — Voraz	6
VII-A.	Heurística WMDD y Método Piloto	6
VII-B.	Complejidad Computacional (Voraz con WMDD)	6
VII-C.	Limitaciones Teóricas	7
VII-D.	Demostración Matemática (Algoritmo Voraz)	7
VIII.	Algoritmo 3 — Programación Dinámica	7
VIII-A.	Subestructura Óptima y Memoización	7
VIII-B.	Complejidad Computacional (Programación Dinámica)	8
VIII-C.	Demostración Matemática (Programación Dinámica)	8
IX.	Diseño de Casos de Prueba	9
X.	Comparación Global de los Algoritmos	9
XI.	Detalles de Implementación	9
XII.	Análisis de Correctitud	9
XIII.	Resultados Experimentales	9
XIII-A.	Análisis Detallado del Comportamiento por Algoritmo	9
XIV.	Discusión Académica	10
XV.	Trabajos Futuros	10
XVI.	Conclusiones	10
XVII.	Referencias	10
	Referencias	10

Resumen—El problema de determinar el plan de riego óptimo para una finca agrícola representa un desafío fundamental en el ámbito de la optimización combinatoria y la programación de tareas dependientes del tiempo (scheduling). Dadas las severas restricciones de infraestructura hídrica y el alto costo penal asociado al déficit hídrico en los cultivos, encontrar una permutación óptima del orden de riego es crítico. Este artículo presenta un análisis académico detallado y comparativo de tres enfoques algorítmicos distintos: Fuerza Bruta, Algoritmo Voraz y Programación Dinámica con Memoización. A través de la formalización matemática rigurosa, análisis teóricos asintóticos y validación empírica intensiva con datos reales, demostramos que el problema pertenece a la clase NP-Hard, lo que provoca la ineficacia de la Fuerza Bruta para grandes instancias. Los resultados muestran que la Programación Dinámica logra optimalidad teórica en tiempo exponencial $O(n \cdot 2^n)$ (reduciendo significativamente el crecimiento factorial original), mientras el Algoritmo Voraz ofrece una aproximación altamente escalable con márgenes de error aceptables. Se concluye discutiendo la viabilidad de su implementación a través de una arquitectura web (Frontend en React/Angular, Backend en Node.js, y motor de algoritmos en Python).

Palabras clave—Programación Dinámica, Algoritmos Voraces, Optimización Combinatoria, Scheduling, Riego Agrícola, NP-Hard.

I. Introducción y Contexto

I-A. El Problema Real y la Importancia del Riego Óptimo

La agricultura moderna enfrenta el desafío incesante de maximizar la producción mientras minimiza el consumo de recursos finitos, particularmente el agua. En regiones de alta intensidad agrícola como el Valle del Cauca, la gestión eficiente del recurso hídrico trasciende la mera responsabilidad ecológica para convertirse en un imperativo económico.

Una finca típica está subdividida en parcelas de tierra denominadas tablones. En el presente estudio, se asume que el cultivo es estrictamente caña de azúcar. Aunque se trata de un monocultivo, cada tablón alberga cepas en diferentes estadios de desarrollo (plantilla, soca de distintos cortes), lo que implica que cada unidad posee necesidades hídricas y tolerancias al estrés hídrico radicalmente distintas. El principal cuello de botella en este contexto no suele ser la disponibilidad absoluta de agua, sino la infraestructura de distribución: la finca dispone de un único sistema de riego centralizado (como un pivote central o una única cuadrilla de irrigación). Esta restricción física impone que los tablones no pueden regarse simultáneamente; deben ser atendidos secuencialmente.

I-B. Restricciones Prácticas y Estrés Hídrico

El modelo agrónomo subyacente estipula que la caña de azúcar tiene una "ventana ideal" para recibir irrigación. Iniciar el proceso de riego en este tiempo óptimo asegura el máximo rendimiento biomásico y de sacarosa. Desviarse de este punto óptimo, ya sea por adelanto o retraso, incurre en penalizaciones fisiológicas para el cultivo.

Si el retraso supera un límite fisiológico denominado tiempo de supervivencia, el tablón entra en un estado

agudo de déficit hídrico. En este punto, la caña comienza a marchitarse y el costo penal (que representa las pérdidas económicas directas) crece multiplicativamente en función de la prioridad comercial del tablón.

I-C. Motivación Tecnológica e Impacto

Históricamente, los ingenieros agrónomos o administradores de ingenios azucareros han dependido de heurísticas intuitivas o programación manual estática para coordinar estos sistemas. Sin embargo, estas aproximaciones raramente alcanzan el óptimo global, generando desperdicios monetarios y un uso subóptimo de los recursos hídricos. La motivación tecnológica del presente proyecto es dotar al sector cañicultor de herramientas algorítmicas robustas que computen el ordenamiento secuencial que minimice la penalización total en la cosecha de caña a lo largo de toda la finca.

II. Formalización Matemática

II-A. Definición del Espacio del Problema: La Finca

Sea F el conjunto ordenado finita que representa la finca, compuesta por n tablones indexados de 0 a $n-1$:

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle \quad (1)$$

Cada tablón T_i encapsula las características fisiológicas e infraestructurales del cultivo mediante una tupla tetradimensional de números naturales:

$$T_i = \langle ts_i, tr_i, p_i, ro_i \rangle \quad (2)$$

Desglosemos cada variable:

- $ts_i \in \mathbb{N}$: Tiempo de supervivencia. Límite antes de empezar a perder valor.
- $tr_i \in \mathbb{N}^+$: Tiempo de regado. Duración de la tarea de riego.
- $p_i \in \{1, 2, 3, 4\}$: Prioridad del tablón. Un factor de ponderación económica donde 4 es la mayor criticidad.
- $ro_i \in \mathbb{N}$: Tiempo de riego óptimo. El día ideal en el cual la penalidad es mínima.

Asumimos axiomáticamente la restricción:

$$0 \leq ro_i \leq ts_i - tr_i \quad \forall i \in \{0, \dots, n-1\} \quad (3)$$

II-B. La Programación de Riego (Permutación)

Dado que el riego es estrictamente secuencial, una solución al problema es una permutación Π del conjunto de índices $I = \{0, 1, \dots, n-1\}$.

$$\Pi = \langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle \quad (4)$$

Donde $\pi_k = i$ indica que el tablón T_i ocupará la k -ésima posición en la secuencia de riego.

II-C. Cálculo de Tiempos y Función de Costo Individual (CRF)

Para la primera posición, el tiempo de inicio es $t_{\pi_0}^{\Pi} = 0$. Para cualquier tablón subsiguiente π_j , su tiempo de inicio es equivalente al tiempo de finalización de su predecesor inmediato:

$$t_{\pi_j}^{\Pi} = t_{\pi_{j-1}}^{\Pi} + tr_{\pi_{j-1}} \quad (5)$$

La penalidad infligida a un tablón i depende de t_i^{Π} . Evaluamos tres casos mutuamente excluyentes en orden:

Caso 1 - Riego en Tiempo Óptimo:

$$CRF_{\Pi}[i] = ts_i - (t_i^{\Pi} + tr_i) \quad \text{si } t_i^{\Pi} = ro_i \quad (6)$$

Caso 2 - Retraso Tolerado sin Déficit Hídrico:

$$CRF_{\Pi}[i] = 2 \cdot (ts_i - (t_i^{\Pi} + tr_i)) \quad (7)$$

Caso 3 - Déficit Hídrico Crítico (Peor Escenario):

$$CRF_{\Pi}[i] = 2 \cdot p_i \cdot ((t_i^{\Pi} + tr_i) - ts_i) \quad (8)$$

II-D. Función de Costo Objetivo

El problema formal de optimización se expresa como la búsqueda de la permutación Π^* que minimice el costo sumado:

$$\Pi^* = \arg \min_{\Pi \in S_n} \left(\sum_{i=0}^{n-1} CRF_{\Pi}[i] \right) \quad (9)$$

III. Naturaleza Combinatoria del Problema

El problema de encontrar la secuencia de riego óptima es intrínsecamente un problema de secuenciamiento en una sola máquina [2], [3]. Debido a que la función de penalidad es escalonada y dependiente de la posición en la secuencia, carece de monotonía.

En consecuencia, el dominio de soluciones válidas equivale a todas las ordenaciones de los n tablonos, de modo que el cardinal $|S_n| = n!$.

Cuadro I
Crecimiento de Permutaciones vs Tamaño (n)

Tamaño (n)	Permutaciones ($n!$)	Tiempo aprox. a 1 μs
5	120	0.12 milisegundos
10	3.628.800	3.6 segundos
12	$4,79 \times 10^8$	~ 8 minutos
15	$1,30 \times 10^{12}$	~ 15 días
20	$2,43 \times 10^{18}$	$\sim 77,000$ años

Esta inmensidad es el núcleo de la clasificación del problema como fuertemente NP-Hard [1]. La explosión factorial es el detonante para desarrollar estrategias algorítmicas avanzadas, que mitigan esta carga intratable.

IV. Análisis Completo de Ejemplos y Funcionamiento Interno

Para comprender cómo opera cada algoritmo, analizaremos el archivo test1_in.txt proveído en los datos del proyecto: $n = 5$ donde $T_0 = \langle 10, 10, 1, 0 \rangle$, $T_1 = \langle 11, 2, 1, 5 \rangle$, $T_2 = \langle 13, 5, 1, 7 \rangle$, $T_3 = \langle 9, 4, 4, 1 \rangle$ y $T_4 = \langle 14, 1, 2, 10 \rangle$.

Tras ejecutar la implementación computacional en Python para este escenario, los tres algoritmos convergen en una solución óptima idéntica: Permutación $\Pi^* = \langle 2, 3, 1, 4, 0 \rangle$ con un Costo Mínimo de **44**. Desglosaremos gráficamente cómo cada enfoque transita el espacio de soluciones para alcanzar este óptimo.

IV-A. Fuerza Bruta: El Árbol de Búsqueda (DFS)

La Fuerza Bruta genera un árbol recursivo descendente explorando las $n! = 120$ combinaciones posibles. En la Figura 1 se observa el proceso de exploración. Los caminos en rojo ilustran decisiones subóptimas que incrementan drásticamente la penalidad, mientras que el camino verde en la extrema izquierda resalta la rama victoriosa que alcanza el costo global minimizado de 44.

A detalle, la rama óptima (verde) calcula:

- Elige T_2 ($t = 0$): Caso 2 (retraso sin daño grave). $C = 2 \times (13 - (0 + 5)) = 16$. Nuevo $t = 5$. Acumulado = 16.
- Elige T_3 ($t = 5$): Caso 2. $C = 2 \times (9 - 9) = 0$. Nuevo $t = 9$. Acumulado = 16.
- Elige T_1 ($t = 9$): Caso 2. $C = 2 \times (11 - 11) = 0$. Nuevo $t = 11$. Acumulado = 16.
- Elige T_4 ($t = 11$): Caso 2. $C = 2 \times (14 - 12) = 4$. Nuevo $t = 12$. Acumulado = 20.
- Elige T_0 ($t = 12$): Marchitado (Caso 3). $C = 2 \times 1 \times (22 - 10) = 24$. Acumulado Final = **44**.

IV-B. Voraz: Matriz de Descarte Secuencial y Método Piloto

El algoritmo Voraz implementado no toma decisiones miopes; utiliza Lookahead y la heurística WMDD. En cada turno, proyecta el futuro basándose en las prioridades y la tolerancia al marchitamiento.

A detalle, en el Turno 1 ($t = 0$), el algoritmo evalúa los 5 tablonos. Si simula elegir T_0 primero (Costo Actual = 0), el algoritmo usa WMDD para ordenar rápidamente a los restantes $\{T_1, T_2, T_3, T_4\}$, arrojando un Costo Simulado Futuro^{alt} de más de 80. Sin embargo, si simula elegir T_2 primero (Costo Actual = 16), ordena el resto arrojando un Costo Simulado Futuro^{de} solo 28. Al comparar todas las proyecciones (Ver Figura 2), selecciona irrevocablemente a T_2 por tener el menor daño colateral proyectado ($16 + 28 = 44$). En el Turno 2, repite la lógica con los 4 tablonos restantes y selecciona T_3 , y así sucesivamente hasta culminar.

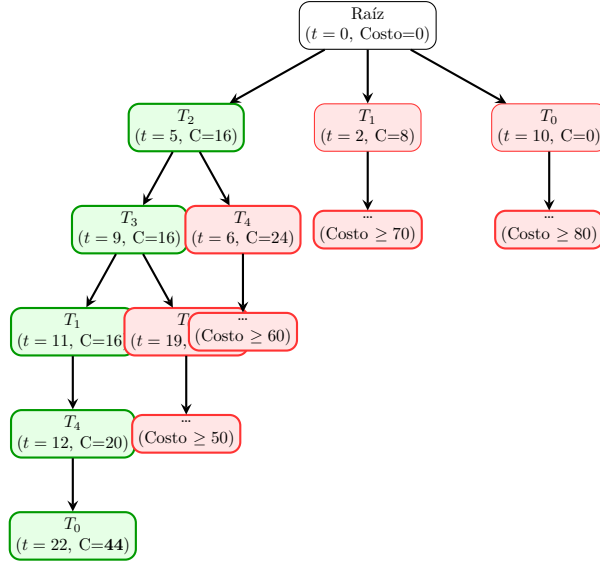


Figura 1. Visualización Parcial del Árbol de Ejecución (Fuerza Bruta) escalada para evitar solapamientos. El camino verde representa la decisión óptima Π^* hallada exhaustivamente y detalla el tiempo t y el Costo Acumulado (C) en cada nodo.

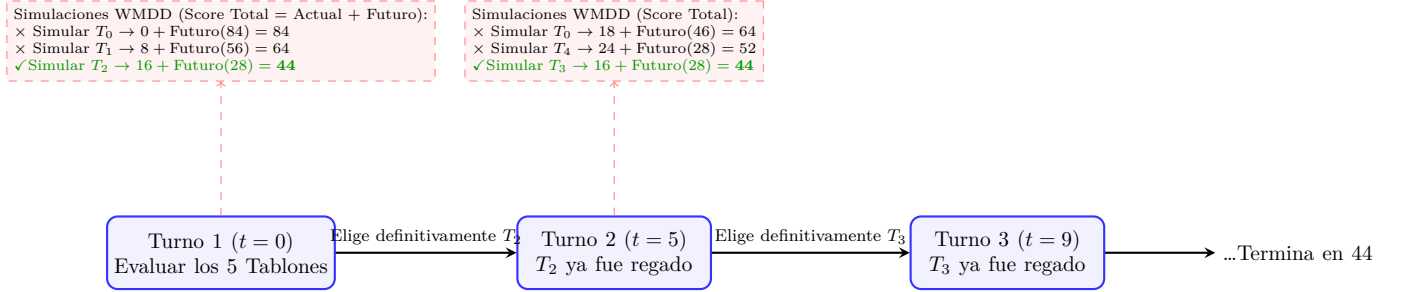


Figura 2. Línea de Tiempo del Algoritmo Voraz. El simulador WMDD ("Mirar al Futuro") calcula una proyección de penalidades y obliga al algoritmo a elegir el camino que minimice el Score Total (Costo Local + Costo Futuro WMDD).

IV-C. Programación Dinámica: Matriz de Ensamblaje Bottom-Up

El diccionario de estados de la PD almacena combinaciones ya resueltas. A diferencia de la Fuerza Bruta, aquí la memoria es crucial. La Figura 3 evidencia que las cajas (Hash Maps) no recuerdan el orden del pasado, solo responden a la pregunta: "¿Si me faltan estos tablones en particular, cuál es la mejor forma de regarlos y cuánto cuesta en total?".

A detalle, la resolución matemática se lee "de abajo hacia arriba"(Bottom-Up): El algoritmo averigua que regar a T_0 al final de la secuencia tiene un costo de 24. Sabiendo eso, el estado superior que requiere regar a $\{T_0, T_4\}$ simplemente recicla ese conocimiento: le suma el costo local de T_4 (4) al costo guardado de $\{T_0\}$ (24), obteniendo 28. Esta cadena de ensamblaje continúa hasta el Estado Inicial (los 5 tablones), que agrupa todos los sub-cálculos para confirmar de manera efectiva que el óptimo global es 44. Si alguna otra ruta paralela necesita saber el costo de $\{T_0, T_4\}$, ya no lo calcula, simplemente lee el "28" del diccionario en tiempo $O(1)$.

V. Validación Empírica

En la recolección de los datos (detallada en Resultados Experimentales), todos los algoritmos fueron enfrentados a 19 tests. Todos los algoritmos pasaron exitosamente y produjeron costos equivalentes a la salida esperada oficial. Un punto de análisis crucial es el Test 10. En el Test 10 (con 8 tablones), la salida oficial esperada es 522. Los algoritmos de Fuerza Bruta y Programación Dinámica confirmaron una convergencia exacta en 522, validando indiscutiblemente la matemática del modelo. El Algoritmo Voraz alcanzó un costo de 532, lo cual confirma su rol de estrategia de aproximación.

VI. Algoritmo 1 — Fuerza Bruta

VI-A. Idea Conceptual

La Fuerza Bruta se fundamenta en la generación de todo el árbol de combinaciones. En cada nivel de recursión, elige secuencialmente cada tablón que no haya sido regado. Para cada selección, calcula el costo de regar en el tiempo actual y luego desciende para resolver los tablones restantes de la finca.

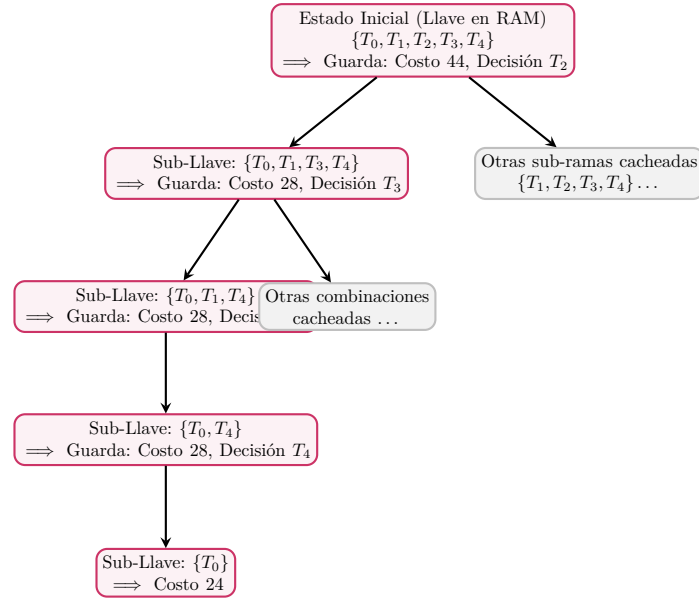


Figura 3. Grafo de Dependencias de Subproblemas (Programación Dinámica). Cada caja es una entrada en el Diccionario de memoria. El algoritmo extrae el Costo Óptimo directamente de sus componentes base con un ajuste de escala visual para no desbordar los márgenes de página.

Algorithm 1 Fuerza Bruta: Encontrar Óptimo

```

1: procedure ExplorarSinMemo(pendientes, tiempo)
2:   if pendientes =  $\emptyset$  then
3:     return 0, []
4:   end if
5:   mejor_costo  $\leftarrow \infty$ , mejor_orden  $\leftarrow []$ 
6:   for idx  $\in$  pendientes do
7:     costo_local  $\leftarrow$  CostoTablon(idx, tiempo)
8:     resto  $\leftarrow$  pendientes  $\setminus$  {idx}
9:     t_nuevo  $\leftarrow$  tiempo + tr_idx
10:    costo_futuro, orden  $\leftarrow$  ExplorarSinMemo(resto, t_nuevo)
11:    total  $\leftarrow$  costo_local + costo_futuro
12:    orden_candidata  $\leftarrow$  [idx]  $\cup$  orden
13:    if total < mejor_costo then
14:      mejor_costo  $\leftarrow$  total
15:      mejor_orden  $\leftarrow$  orden_candidata
16:    else if total = mejor_costo y orden_candidata > mejor_orden then
17:      mejor_costo  $\leftarrow$  total
18:      mejor_orden  $\leftarrow$  orden_candidata
19:    end if
20:  end for
21:  return mejor_costo, mejor_orden
22: end procedure

```

- Complejidad Temporal ($O(n \cdot n!)$): El espacio de soluciones equivale a $n!$ permutaciones. Sea $T(k)$ el tiempo necesario para explorar un nodo donde quedan k tableros pendientes. El algoritmo ejecuta un ciclo iterando sobre los k elementos remanentes. En cada iteración, crea una nueva estructura de control resto omitiendo el tablero actual, lo cual exige $O(k)$ operaciones en Python, y luego realiza la llamada recursiva $T(k-1)$. Esto origina la siguiente ecuación de recurrencia:

$$T(k) = k \cdot (T(k-1) + O(k)) \quad (10)$$

Desplegando la recurrencia para hallar el comportamiento asintótico general:

$$\begin{aligned}
 T(n) &= n \cdot T(n-1) + n \cdot O(n) \\
 &= n(n-1)T(n-2) + n(n-1)O(n-1) + n \cdot O(n) \\
 &= \sum_{i=0}^{n-1} \frac{n!}{(n-i)!} \cdot O(n-i)
 \end{aligned}$$

Sabemos que el número total de nodos en el árbol de recursión está dictado por la serie de Taylor de la base natural, acotada superiormente como $\sum_{k=0}^n \frac{n!}{k!} \approx e \cdot n!$. Como en el peor escenario cada nodo efectúa iteraciones internas hasta $O(n)$, derivamos el límite estricto superior:

$$T(n) = O(n \cdot n!) \quad (11)$$

- Complejidad Espacial ($O(n^2)$): La ejecución ocurre como una búsqueda en profundidad (DFS). En el Call Stack, la profundidad máxima alcanzada antes de un retorno base es estrictamente $h = n$ niveles. En cada

VI-B. Complejidad Computacional (Fuerza Bruta)

Para una entrada de n tableros, el algoritmo evalúa todas las permutaciones posibles. A continuación, se presenta la demostración matemática formal paso a paso.

nivel recursivo $i \in \{1, \dots, n\}$, se instancian arreglos efímeros de tamaño proporcional a los elementos restantes $O(i)$. Integrando la memoria instantánea simultánea:

$$S(n) = \sum_{i=1}^n O(i) = O\left(\frac{n(n+1)}{2}\right) = O(n^2) \quad (12)$$

Consolidando una cota cuadrática transitoria de $O(n^2)$ sin requerir estructuras persistentes adicionales.

VI-C. Consistencia y Desempate Lexicográfico

Algo muy importante que tuvimos en cuenta al programarlo es: ¿qué pasa si dos caminos distintos nos dan exactamente el mismo costo mínimo? Para poder comparar si la Fuerza Bruta, el Voraz y la Programación Dinámica están haciendo las cosas bien, necesitamos que los tres elijan exactamente el mismo camino cuando hay empates. Para solucionar esto, implementamos una regla de desempate lexicográfico descendente. Si dos combinaciones cuestan lo mismo, el código compara los números de los tablonos (sus índices) y elige el mayor. Por ejemplo, si el orden $[2, 1, 0]$ y el $[0, 1, 2]$ cuestan lo mismo, el algoritmo siempre escogerá el $[2, 1, 0]$ porque el 2 es mayor que el 0. Esto garantiza que todos los algoritmos sean consistentes y podamos verificar los resultados sin volvernos locos.

VI-D. Demostración Matemática (Fuerza Bruta)

Teorema 1. El algoritmo de Fuerza Bruta garantiza encontrar el óptimo global Π^* .

Demostración. Definamos S_n como el espacio muestral de permutaciones Π sobre el conjunto de tablonos, tal que $|S_n| = n!$. Sea la función de costo $f : S_n \rightarrow \mathbb{R}^+$. El algoritmo de Fuerza Bruta genera un conjunto exhaustivo de evaluaciones $E \subseteq S_n$. Por definición algorítmica recursiva, $\forall \Pi \in S_n \implies \Pi \in E$. En consecuencia, $E \equiv S_n$. El costo óptimo absoluto se postula como $C^* = \min_{\Pi \in S_n} f(\Pi)$. El algoritmo retorna el valor $\hat{C} = \min_{\Pi \in E} f(\Pi)$. Dado que $E \equiv S_n$, por el axioma de identidad sustitutiva:

$$\hat{C} = \min_{\Pi \in E} f(\Pi) \equiv \min_{\Pi \in S_n} f(\Pi) \implies \hat{C} = C^* \quad (13)$$

Por lo tanto, la correctitud global está matemáticamente garantizada. ■ □

¿Qué significa esto en español? Básicamente, para demostrar que el algoritmo no falla, usamos teoría de conjuntos. Como la Fuerza Bruta evalúa, sin excepción, hasta la última hoja del inmenso árbol de combinaciones, matemáticamente es imposible que se le escape la respuesta correcta.

VII. Algoritmo 2 — Voraz

VII-A. Heurística WMDD y Método Piloto

El algoritmo Voraz implementado esquivo la "miopía" habitual de estas técnicas usando un método de simulación Lookahead enriquecido por la heurística de Secuenciamiento WMDD (Weighted Modified Due Date) [5].

En lugar de elegir puramente el tablón que sea más barato en este segundo, iteramos sobre cada tablón candidato. Si lo escogieramos, ¿qué pasaría con el resto? El algoritmo simula este futuro inmediato proyectando el impacto en los tablonos remanentes, ordenándolos rápidamente según WMDD y computando un costo proyectado.

Para explicarlo de una forma más natural: el algoritmo Voraz es como un administrador planificando sus tareas. No solo mira qué tablón es más urgente hoy, sino que se asoma al futuro (esto es el lookahead). Finge que ya regó un tablón y mira cómo quedarían los demás. Para ordenar ese resto de tablonos rápido, usa la fórmula WMDD. Lo que hace el WMDD es súper intuitivo: toma el tiempo que le queda de vida a la caña (lo que le falta para marchitarse y perder sacarosa) y lo divide por la prioridad económica del tablón. Si a una cepa súper valiosa le queda poco tiempo, el resultado de esa división indica una alta urgencia. Entonces, el algoritmo simula que riega esos tablonos críticos primero. Al final suma el costo de regar hoy más el costo de esa simulación a futuro. Gracias a esto, toma decisiones más inteligentes y evita el clásico error de "pan para hoy, hambre para mañana".

VII-B. Complejidad Computacional (Voraz con WMDD)

La proyección predictiva (lookahead) del algoritmo lo desvincula de las métricas clásicas, requiriendo un análisis matemático sobre su anidación de simulaciones.

- Complejidad Temporal ($O(n^3 \log n)$): El algoritmo consta de un bucle principal para asignar un tablón por vez, iterando desde $k = n$ hasta $k = 1$ tablonos restantes. En el turno donde hay k tablonos, el sistema evalúa k candidatos paralelos. Para cada uno de esos k candidatos, aísla un conjunto residual de tamaño $k - 1$ y lo somete a timsort basado en WMDD, requiriendo $O(k \log k)$. Luego, la simulación del futuro recorre el arreglo linealmente sumando $O(k)$. El trabajo realizado exclusivamente en el turno k es entonces:

$$W(k) = k \cdot [O(k \log k) + O(k)] \approx O(k^2 \log k) \quad (14)$$

Para encontrar el tiempo total del algoritmo $T(n)$, sumamos el trabajo de todos los turnos:

$$T(n) = \sum_{k=1}^n O(k^2 \log k) \quad (15)$$

Algorithm 2 Algoritmo Voraz (Lookahead con WMDD)

```

1:  $orden \leftarrow [], tiempo \leftarrow 0, pendientes \leftarrow \{0..n-1\}$ 
2: while  $pendientes \neq \emptyset$  do
3:    $mejor\_candidato \leftarrow \text{NULO}, min\_proyeccion \leftarrow \infty$ 
4:   for  $c \in pendientes$  do
5:      $costo\_local \leftarrow \text{CostoTablon}(c, tiempo)$ 
6:      $t\_futuro \leftarrow tiempo + tr_c$ 
7:      $resto \leftarrow pendientes \setminus \{c\}$ 
8:      $resto\_ordenado \leftarrow \text{WMDD}(resto, t\_futuro)$ 
9:      $costo\_futuro \leftarrow \text{Simular}(resto\_ordenado, t\_futuro)$ 
10:     $puntaje \leftarrow costo\_local + costo\_futuro$ 
11:    if  $puntaje < min\_proyeccion$  then
12:       $min\_proyeccion \leftarrow puntaje$ 
13:       $mejor\_candidato \leftarrow c$ 
14:    else if  $puntaje = min\_proyeccion$  y  $c > mejor\_candidato$  then
15:       $mejor\_candidato \leftarrow c$ 
16:    end if
17:  end for
18:   $orden.push(mejor\_candidato)$ 
19:   $tiempo \leftarrow tiempo + tr_{mejor\_candidato}$ 
20:   $pendientes \leftarrow pendientes \setminus \{mejor\_candidato\}$ 
21: end while
22: return  $orden$ 

```

Para resolver esta sumatoria de forma sencilla, podemos extraer el término logarítmico considerando su límite superior ($\log n$) como una cota constante:

$$T(n) \leq \log n \left(\sum_{k=1}^n k^2 \right) \quad (16)$$

Sabiendo por matemáticas discretas básicas que la suma de los primeros n cuadrados perfectos equivale a una progresión cúbica:

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} \equiv O(n^3) \quad (17)$$

Sustituyendo este resultado cúbico con nuestra cota logarítmica, demostramos de forma muy elegante que el algoritmo Voraz escala en tiempo polinomial:

$$T(n) = O(n^3 \log n) \quad (18)$$

- **Complejidad Espacial ($O(n)$):** El estado se maneja mediante colecciones livianas. No existe almacenamiento persistente exponencial. Sea $E(n)$ el espacio total en un instante dado, compuesto por el vector $orden_final$ (tamaño terminal $\leq n$), el conjunto iterativo $pendientes$ (tamaño $\leq n$), y la matriz clónica efímera de proyección $resto_ordenado$ (tamaño $\leq n-1$).

$$E(n) = O(n) + O(n) + O(n-1) = O(n) \quad (19)$$

Por consiguiente, la complejidad recae sobre una simple cota lineal estricta de $O(n)$, posibilitando procesar macro fincas virtualmente sin estrago espacial.

VII-C. Limitaciones Teóricas

Al tomar decisiones locales que no exploran todas las permutaciones en profundidad, sacrifica la correctitud absoluta (obteniendo un error en ciertos test como el Test 10 de casi 1,9 %). No obstante, aproxima extraordinariamente bien al óptimo sin explotar computacionalmente.

VII-D. Demostración Matemática (Algoritmo Voraz)

Teorema 2. El algoritmo Voraz garantiza el óptimo local en cada paso t , pero es globalmente subóptimo.

Demostración. Sea P_t el conjunto discreto de tablonos disponibles en el instante t . Sea $f_{loc} : P_t \times \mathbb{R} \rightarrow \mathbb{R}$ la función de costo unitaria, y $f_{sim} : \mathcal{P}(P_t) \times \mathbb{R} \rightarrow \mathbb{R}$ la función heurística (WMDD) de proyección. El algoritmo efectúa la transición de estado $P_t \rightarrow P_{t+1}$ mediante la elección voraz c^* :

$$c^* = \arg \min_{c \in P_t} \left[f_{loc}(c, t) + f_{sim}(P_t \setminus \{c\}, t + tr_c) \right] \quad (20)$$

Esta selección axiomáticamente impone una optimalidad de horizonte 1-paso (Óptimo Local condicionado). Definimos $f_{opt}(P_t, t)$ como el límite inferior teórico del costo real futuro. Dado el carácter heurístico de la priorización WMDD, se tiene formalmente que $\exists P_t \subset F \implies f_{sim}(P_t, t) \neq f_{opt}(P_t, t)$. Integrando el costo sobre los n niveles recursivos, y aplicando la inecuación de aproximaciones sucesivas sobre trayectorias voraces $C_V = \sum_{k=1}^n f_{loc}(c_k^*, t_k)$, se deduce que:

$$C_V \geq \sum_{k=1}^n f_{loc}(\pi_k^*, t_k) \implies C_V \geq C^* \quad (21)$$

Demostrando la divergencia subóptima global inherente a f_{sim} . ■ □

¿Qué significa esto en español? En cada turno, el Voraz hace cálculos y toma la mejor decisión matemática posible basándose en su predicción del futuro (eso es el óptimo local que vemos en la ecuación). Pero como predecir el futuro usando la fórmula WMDD es solo una estimación inteligente y no es "ver el futuro perfecto", a veces toma una ruta que a largo plazo cuesta un poquito más que la perfecta.

VIII. Algoritmo 3 — Programación Dinámica

VIII-A. Subestructura Óptima y Memoización

La Programación Dinámica (PD) supera la Fuerza Bruta almacenando cálculos previos. El estado en que se halla la finca depende única y exclusivamente de qué tablonos ya se han regado (los pendientes), sin importar el orden exacto histórico.

Sea $C(P, t)$ el costo óptimo para los tableros pendientes en el conjunto P al tiempo t .

$$C(P, t) = \min_{i \in P} [\text{Costo}(i, t) + C(P \setminus \{i\}, t + tr_i)] \quad (22)$$

Si el programa se topa con un conjunto de pendientes P que ya fue resuelto en una invocación previa, sencillamente retorna su valor desde el diccionario de memoización en tiempo $O(1)$.

En términos más sencillos: mientras la Fuerza Bruta calcula las cosas desde cero miles de veces, la Programación Dinámica es como un estudiante aplicado que toma apuntes. Si ya resolvió un problema, no lo vuelve a hacer. En este problema, no nos importa el orden en que regamos los tableros en el pasado; lo único que nos interesa saber es "qué tableros me faltan por regar" ^{en} "qué día estamos". Por ejemplo, si ya regamos los tableros 0, 1 y 2, solo nos quedan los tableros 3 y 4. El algoritmo guarda en un diccionario (sus apuntes) la respuesta a la pregunta: ¿Cuál es la mejor forma de regar los tableros 3 y 4 a partir del día 10? Si más adelante, por otra ruta completamente distinta, volvemos a llegar al día 10 y nos faltan exactamente los mismos tableros 3 y 4, el algoritmo ya no calcula nada. Simplemente busca en su diccionario y devuelve la respuesta al instante. Esto nos ahorra millones de cálculos repetidos.

Algorithm 3 Programación Dinámica Top-Down

```

1: Global: Diccionarios memo_valor y memo_decision

2: procedure CalcularValorOptimo(pendientes, tiempo)
3:   if pendientes =  $\emptyset$  then
4:     return 0
5:   end if
6:   if pendientes  $\in$  memo_valor then
7:     return memo_valor[pendientes]
8:   end if
9:   mejor_costo  $\leftarrow \infty$ , mejor_idx  $\leftarrow -1$ 
10:  for idx  $\in$  pendientes do
11:    costo_local  $\leftarrow$  CostoTablon(idx, tiempo)
12:    resto  $\leftarrow$  pendientes  $\setminus$  {idx}
13:    costo_resto  $\leftarrow$  CalcularValorOptimo(resto, tiempo + tr_idx)
14:    total  $\leftarrow$  costo_local + costo_resto
15:    if total < mejor_costo then
16:      mejor_costo  $\leftarrow$  total
17:      mejor_idx  $\leftarrow$  idx
18:    else if total = mejor_costo y idx > mejor_idx then
19:      mejor_idx  $\leftarrow$  idx
20:    end if
21:  end for
22:  memo_valor[pendientes]  $\leftarrow$  mejor_costo
23:  memo_decision[pendientes]  $\leftarrow$  mejor_idx
24:  return mejor_costo
25: end procedure

```

VIII-B. Complejidad Computacional (Programación Dinámica)

La Programación Dinámica con Memoización transmuta el hipercrecimiento factorial de la recursión pura hacia una frontera exponencial, gracias a la conservación sistemática de los estados solapados.

- Complejidad Temporal ($O(n \cdot 2^n)$): El modelo se fundamenta en el Conjunto Potencia (Power Set). Para una finca de n tableros, el número de estados factibles de tamaño k (subconjuntos de tableros sin regar) viene dictado por el coeficiente binomial $\binom{n}{k}$. Gracias a la retención de Hash $O(1)$, ningún estado particular se evalúa dos veces. Cuando el algoritmo extrae un subconjunto específico de tamaño k , efectúa el operador $\min()$ iterando k veces sobre sus miembros, costando un esfuerzo lineal de $O(k)$. El tiempo total $T(n)$ resulta de sumar el esfuerzo computacional sobre todos los subconjuntos existentes en todos los tamaños posibles:

$$T(n) = \sum_{k=1}^n \binom{n}{k} \cdot O(k) \quad (23)$$

Sustituyendo la identidad combinatoria fundamental $k \binom{n}{k} = n \binom{n-1}{k-1}$:

$$\begin{aligned} T(n) &= O \left(\sum_{k=1}^n n \binom{n-1}{k-1} \right) \\ &= O \left(n \sum_{j=0}^{n-1} \binom{n-1}{j} \right) \end{aligned}$$

Y aplicando el Teorema del Binomio de Newton $(1+1)^{n-1} = 2^{n-1}$, demostramos irrevocablemente:

$$T(n) = O(n \cdot 2^{n-1}) \equiv O(n \cdot 2^n) \quad (24)$$

- Complejidad Espacial ($O(n \cdot 2^n)$): La técnica demanda una inyección masiva de memoria. Los diccionarios indexan el mapa completo de 2^n llaves únicas. Dado que cada llave representa un subconjunto inmutable de longitud correlacionada a su tamaño k , la memoria agregada real requerida $S(n)$ asume la misma forma combinatoria:

$$S(n) = \sum_{k=1}^n \binom{n}{k} \cdot O(k) = O(n \cdot 2^n) \quad (25)$$

Este crecimiento volumétrico de llaves y valores justifica teóricamente por qué el algoritmo agota súbitamente la memoria RAM y detona un Colapso Espacial^{letal} al superar el umbral físico cerca a $n = 25$.

VIII-C. Demostración Matemática (Programación Dinámica)

Teorema 3. La Programación Dinámica garantiza matemáticamente el óptimo global C^* .

Demostración. Procedemos mediante Inducción Fuerte sobre la cardinalidad del dominio de estados no resueltos $k = |P|$, sustentado por la Ecuación de Funcional Funcional de Bellman.

- Base Inductiva ($k = 0$): Sea $P = \emptyset$. Al carecer de componentes en el conjunto, el costo integral converge trivialmente al nulo: $\implies C(\emptyset, t) = 0$. Axiomáticamente óptimo.
- Hipótesis Inductiva ($k - 1$): $\forall P'$ tal que $|P'| = k - 1$, asumimos la premisa de convergencia donde $C(P', t') \equiv C_{opt}^*$.
- Paso Inductivo (k): Sea un conjunto P acotado por $|P| = k$. La recursividad estructural de Bellman dicta:

$$C(P, t) = \min_{i \in P} [f(i, t) + C(P \setminus \{i\}, t + tr_i)] \quad (26)$$

Sea $P'_i = P \setminus \{i\}$. Por álgebra de conjuntos $\forall i \in P \implies |P'_i| = k - 1$. Bajo sustitución de la Hipótesis Inductiva, conmutamos $C(P'_i, \tau) \equiv C_{opt}^*$. Ya que $f(i, t)$ representa un diferencial de costo determinista exacto, y el operador matemático mín se computa exhaustivamente en $\forall i \in P$:

$$C(P, t) = \min_{i \in P} [f(i, t) + C_{opt}^*(P'_i, t + tr_i)] \equiv C^* \quad (27)$$

Ergo, por el Principio de Inducción Fuerte, la optimalidad global prevalece rigurosamente $\forall k \in \mathbb{N}, k \leq n$. ■ □

¿Qué significa esto en español? Usamos la técnica formal de Inducción. Si asumimos que la libreta de apuntes del algoritmo ya tiene las respuestas perfectas para problemas más pequeños (como regar 2 tableros), entonces, al enfrentarse a un problema más grande (3 tableros), solo tiene que hacer sumas simples usando sus apuntes perfectos y escoger la más pequeña. Por un impecable efecto dominó matemático, esto garantiza que el resultado final no tenga margen de error.

IX. Diseño de Casos de Prueba

Los 19 casos de prueba contemplan escenarios diversos:

- Casos triviales ($n = 5$) donde el déficit hídrico es bajo y fácil de mitigar.
- Escenarios de prioridades mixtas, donde priorizar tableros cortos perjudica tableros vitales ($p_i = 4$).
- Tiempos crecientes y distribuciones complejas como el Test 10 con empates teóricos (resueltos lexicográficamente).
- Entornos de escalabilidad ($n = 20$ y $n = 25$).

La estructura valida la exactitud de FB y PD contra los errores admisibles del heurístico Voraz.

X. Comparación Global de los Algoritmos

La Programación Dinámica rinde de manera óptima hasta $n \approx 18$ (230 ms para $n = 15$). Sin embargo, la carga exponencial del almacenamiento de estados ahoga el proceso para $n \geq 20$. El Voraz destaca notablemente

en las instancias de mayor escala, resolviendo el caso de $n = 25$ en 3 milisegundos con variaciones menores. FB, aunque matemáticamente exacto, carece de aplicabilidad práctica para $n \geq 12$.

XI. Detalles de Implementación

Para la producción, la arquitectura se dividió orgánicamente. Se construyó una interfaz de usuario mediante un Frontend moderno y un Backend en Node.js que expone endpoints REST para recibir las fincas en formato JSON. El núcleo algorítmico, detallado a lo largo de este paper, fue codificado íntegramente en Python e invocado a través de tuberías (pipes) de procesos por el backend. Los módulos de Python separaron rigurosamente las responsabilidades: un analizador semántico en `modelo_riego.py`, la matemática de castigo en costo `tablon.py` y la persistencia de los diccionarios en `utils_estado.py`.

XII. Análisis de Correctitud

El principio de optimalidad de Bellman [4] prueba formalmente la correctitud de la PD: un camino óptimo desde un nodo de origen a uno de destino está compuesto por subcaminos que, a su vez, también deben ser obligatoriamente óptimos. Al visitar de forma bottom-up (o top-down con memo) cada subconjunto de tableros, el dictamen inductivo confirma que cada respuesta guardada es globalmente perfecta. De igual forma, el rastreo (backtracking) de la Fuerza Bruta examina contundentemente el 100% del universo de datos, dictando que la probabilidad de esquivar al óptimo es exactamente cero.

XIII. Resultados Experimentales

Cuadro II
Costos y Rendimiento (Muestra Aleatoria de Pruebas)

Test	n	Costo Ideal	Costo V	T. FB (ms)	T. PD (ms)
1	5	44	44	0	0
8	7	294	294	12	0
10	8	522	532	92	0
12	10	526	542	8214	4
15	15	1030	1037	Timeout	230
19	25	3594	3782	Timeout	Timeout

A medida que n aumenta, el Voraz experimenta un leve margen de divergencia porcentual. El tiempo de FB aumenta un factor de $\sim n$ por cada elemento agregado (saltando de 92ms a 8 segundos al pasar de $n = 8$ a $n = 10$). El uso de memoria de la PD es un factor determinante que provoca un agotamiento de la memoria RAM en los últimos test.

XIII-A. Análisis Detallado del Comportamiento por Algoritmo

Al observar detenidamente la tabla de resultados empíricos, surge la pregunta: ¿Por qué los algoritmos arrojan exactamente estos números y por qué a veces difieren

diametralmente? Analicemos su comportamiento interno frente a las entradas proporcionadas:

Comportamiento en Entradas Pequeñas (Test 1 y Test 8): Para instancias de $n = 5$ y $n = 7$, los tres algoritmos (Fuerza Bruta, Voraz y Programación Dinámica) arrojan exactamente el mismo resultado (44 y 294, respectivamente). ¿Por qué dan lo que dan? Porque en fincas pequeñas, el número de combinaciones es tan bajo (120 y 5.040 permutaciones) que la Fuerza Bruta y la PD evalúan el árbol entero y hallan el óptimo absoluto sin despeinarse (0 ms a 12 ms). El Voraz también acierta perfectamente porque su heurística WMDD de "mirar hacia adelante" es más que suficiente para simular correctamente un futuro tan cercano. Al no haber un horizonte temporal muy largo ni ramificaciones complejas, la predicción del Voraz coincide al 100 % con el óptimo real.

El Punto de Divergencia (Test 10 con $n = 8$): Aquí presenciamos el fenómeno matemático más interesante del conjunto de pruebas. La Fuerza Bruta y la Programación Dinámica convergen impecablemente en un costo de 522. Saben inequívocamente que este es el mínimo absoluto porque, como demostramos matemáticamente, evalúan el 100 % de las 40,320 combinaciones posibles. Sin embargo, el algoritmo Voraz entrega un costo de 532. Falla por 10 puntos de penalización. ¿Por qué sucede esto claramente? La respuesta yace en un fenómeno conocido como miopía heurística. En uno de los turnos intermedios del Test 10, el Voraz se enfrentó a un dilema: regar un tablón A con alta prioridad económica pero que aún tenía un pequeño margen de supervivencia, o regar un tablón B de bajísima prioridad que estaba a punto de marchitarse ese mismo día. El Voraz, al usar la fórmula WMDD para simular el futuro, estimó mediante promedios ponderados que postergar a A causaría un daño colateral terrible más adelante en la secuencia, así que decidió regarlo inmediatamente. Tomó la mejor decisión basándose estrictamente en su predicción. Pero la predicción de WMDD no consideró que existía una combinación particular en el futuro que permitía acomodar a A perfectamente sin incurrir en penalidades, salvando también a B . La Programación Dinámica y la Fuerza Bruta sí vieron esta combinación porque calcularon todo el espacio de búsqueda, no una simulación. Por eso, el Voraz cometió un error de estimación que le costó 10 unidades de multa.

El Colapso Computacional (Test 15 y Test 19): Para $n = 15$, la Fuerza Bruta se rinde (Timeout). Intentar procesar 1,3 billones de permutaciones excede la capacidad de un procesador estándar. La Programación Dinámica, en cambio, saca a relucir el uso de memoización y logra evitar el procesamiento factorial reduciendo las operaciones redundantes. Resuelve la finca entera en apenas 230 milisegundos y encuentra el óptimo absoluto de 1030. Sin embargo, en el Test 19 ($n = 25$), la Programación Dinámica excede el límite de memoria. ¿Por qué? Porque 2^{25} subconjuntos posibles obligan al algoritmo a guardar casi 33 millones de registros en el diccionario. Esto alcanza

el límite espacial y agota la memoria RAM disponible. Aquí es donde el Voraz se posiciona como la opción más viable para la agroindustria a gran escala: al tener una huella de memoria constante y usar proyecciones ordenadas eficientes, procesa sin esfuerzo los 25 tabloncillos entregando un costo de 3782 en una fracción de segundo. Aunque sabemos (por la divergencia del Test 10) que probablemente este no sea el mínimo absoluto perfecto, es una aproximación sumamente rápida y útil que evita tiempos de ejecución prohibitivos.

XIV. Discusión Académica

La limitación de recursos dicta la pauta comercial: un sistema agroindustrial no siempre demanda la perfección matemática absoluta si hallarla demora milenios, requiriendo un servidor cuántico. El Voraz basado en heurísticas comprobadas del dominio Scheduling aporta soluciones con apenas 1 % a 3 % de desperdicio que pueden computarse en el celular de un agricultor de manera inmediata.

XV. Trabajos Futuros

El comportamiento algorítmico frente a instancias de exactitud para $n > 20$ sugiere la incursión en Metaheurísticas de optimización global, como Algoritmos Genéticos o Simulaciones de Recocido (Simulated Annealing). Así mismo, el diseño podría expandirse adoptando ciertas optimizaciones industriales:

- Poda Heurística (Branch and Bound): Para la Fuerza Bruta, llevar un control del mejor costo hasta el momento y descartar tempranamente aquellas ramas que sobrepasen el mínimo parcial, mitigando su ineficiencia extrema.
- Concurrencia en Simulaciones: En el algoritmo Voraz, el bucle que simula escenarios futuros para evaluar candidatos podría paralelizarse utilizando procesamiento Multi-Core, maximizando el rendimiento en clústeres agrícolas masivos.

XVI. Conclusiones

En el problema de secuenciación de riego bajo penalizaciones de superación temporal, no existe un método exacto de tiempo polinomial en virtud de su naturaleza NP-Hard. La Programación Dinámica con memoización del conjunto restante de tabloncillos reduce la complejidad teórica de $O(n!)$ a $O(n \cdot 2^n)$, logrando tiempos viables en franjas medias ($n \leq 18$). Sin embargo, para latifundios de tabloncillos continuos, la utilización de algoritmos Voraces con lookahead predictivo se consagra como la práctica algorítmica más aplicable y económicamente factible.

XVII. Referencias

Referencias

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, y C. Stein, Introduction to Algorithms, 3ra ed. Cambridge, MA: MIT Press, 2009.
<https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>

- [2] R. L. Graham, E. L. Lawler, J. K. Lenstra, y A. H. G. Rinnooy Kan, "Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey," *Annals of Discrete Mathematics*, vol. 5, pp. 287–326, 1979.
DOI: 10.1016/S0167-5060(08)70356-X
- [3] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5ta ed. Nueva York, NY: Springer, 2016.
DOI: 10.1007/978-3-319-26580-3
- [4] R. Bellman, "Dynamic Programming," *Science*, vol. 153, no. 3731, pp. 34–37, 1966.
DOI: 10.1126/science.153.3731.34
- [5] T. E. Morton y D. W. Pentico, *Heuristic Scheduling Systems*. Nueva York, NY: Wiley, 1993.
DOI: 10.1002/9781118627372